DevOps Projects Complete Guide

How to run, test, and understand every project

CI/CD Pipeline | Monitoring Stack | Terraform Infrastructure | Kubernetes Microservices

Suraj Maitra

Cloud and DevOps Engineer

github.com/Srj0210

Table of Contents

Chapter 1

CI/CD Pipeline with GitHub Actions

A 7-stage automated pipeline that builds, tests, scans, and ships a containerized application.

What This Project Is

This is a complete CI/CD pipeline built on GitHub Actions. Every time you push code to the main branch, it automatically runs through seven stages: code linting, unit tests, Dockerfile linting, Docker image build, security scanning with Trivy, integration testing, and finally pushing the image to GitHub Container Registry.

The application itself is a simple Flask API with three endpoints. The point is not the app. The point is the pipeline around it.

What You Need Before Starting

- Docker installed on your machine
- Python 3.11 installed
- A GitHub account (the pipeline runs on GitHub for free)
- Git installed for cloning and pushing

File Structure

```
cicd-pipeline/
|-- .github/
| +-- workflows/
| +-- pipeline.yml # The CI/CD workflow (7 stages)
|-- tests/
| +-- test_app.py # 5 unit test cases
|-- app.py # Flask API (3 endpoints)
|-- Dockerfile # Multi-stage production build
|-- requirements.txt # Python dependencies
+-- README.md
```

Step 1: Clone and Explore

```
git clone https://github.com/Srj0210/cicd-pipeline.git
cd cicd-pipeline
```

Take a minute to open each file and read through it. The project is small on purpose so you can understand every line.

Step 2: Run the Application Locally

First, install the dependencies and run the app without Docker to make sure the code works:

```
pip install -r requirements.txt
python app.py
```

The app will start on port 5000. Open another terminal and test it:

```
curl http://localhost:5000/
curl http://localhost:5000/health
curl http://localhost:5000/api/info
```

You should see JSON responses from each endpoint. The health endpoint returns a simple status ok. The root endpoint returns the service version and environment. The info endpoint returns metadata about the project.

Step 3: Run the Tests

```
pip install pytest pytest-cov
pytest tests/ -v --cov=. --cov-report=term-missing
```

This runs all 5 test cases and shows you code coverage. Every test should pass. The tests check each endpoint for correct status codes, JSON structure, and response values. The -v flag shows verbose output so you can see each test individually.

Step 4: Build the Docker Image

```
docker build -t cicd-demo:latest .
docker images | grep cicd-demo
```

The Dockerfile uses a multi-stage build. The first stage installs Python dependencies. The second stage copies only the installed packages and application code into a slim image. This keeps the final image small and removes unnecessary build tools.

> *Note: The image runs as a non-root user called appuser. This is a security best practice. If someone compromises the app, they get limited permissions instead of root access.*

Step 5: Run the Container

```
docker run -d --name cicd-test -p 5000:5000 cicd-demo:latest
curl http://localhost:5000/health
```

If you get a healthy response, the container is working. This is essentially what the integration test stage in the pipeline does automatically.

```
# Stop and remove when done
docker stop cicd-test
docker rm cicd-test
```

Step 6: The Pipeline (GitHub Actions)

Push this code to your GitHub repo and the pipeline triggers automatically. Go to the Actions tab on your repository to watch it run. Here is what each stage does:

Stage	What It Does	Tool Used
1. Code Quality	Checks code style and quality score	Flake8, Pylint
2. Unit Tests	Runs all test cases with coverage	Pytest
3. Dockerfile Lint	Checks Dockerfile best practices	Hadolint
4. Build Image	Creates multi-stage Docker image	Docker Buildx
5. Security Scan	Scans image for vulnerabilities	Trivy
6. Integration Test	Starts container, hits real endpoints	curl, Docker
7. Push to GHCR	Pushes image to container registry	GHCR

How to Verify the Pipeline Works

- Push any change to main branch
- Go to github.com/Srj0210/cicd-pipeline then click the Actions tab
- You should see the pipeline running with all 7 stages
- Green checkmarks mean everything passed
- Click any stage to see detailed logs

Note: The Push to GHCR stage only runs on the main branch, not on pull requests. This prevents incomplete code from being published.

Chapter 2
Docker Monitoring Stack

Prometheus, Grafana, Node Exporter, cAdvisor, and Alertmanager running with a single command.

What This Project Is

This is a complete monitoring and alerting stack that runs entirely in Docker. Prometheus collects metrics every 15 seconds from the host machine, all running containers, and a demo application. Grafana displays everything on a pre-built dashboard. Alertmanager fires alerts when things go wrong. You start it with one command and everything is wired together automatically.

What You Need

- Docker and Docker Compose installed
- At least 2 GB of free RAM (the stack runs 6 containers)

File Structure

```
monitoring-stack/
|-- docker-compose.yml # Orchestrates 6 services
|-- prometheus/
| |-- prometheus.yml # Scrape targets and intervals
| +-- alert-rules.yml # 7 alert rules
|-- grafana/
| |-- provisioning/
| | |-- datasources/
| | | +-- prometheus.yml # Auto-configures Prometheus
| | +-- dashboards/
| | +-- dashboards.yml # Dashboard provider config
| +-- dashboards/
| +-- infrastructure.json # Pre-built 8-panel dashboard
|-- alertmanager/
| +-- alertmanager.yml # Alert routing config
+-- demo-app/
|-- app.py # Python app with Prometheus metrics
+-- Dockerfile
```

Step 1: Clone and Start

```
git clone https://github.com/Srj0210/monitoring-stack.git
cd monitoring-stack
docker-compose up -d
```

Docker will pull the images (first time takes 2-3 minutes), build the demo app, and start all 6 containers. Run this to verify everything is running:

```
docker-compose ps
```

You should see 6 containers all showing Up status.

Step 2: Open Grafana

Open your browser and go to:

```
http://localhost:3000
```

Login credentials:

- Username: **admin**
- Password: **devops123**

After logging in, the dashboard is already there. You do not need to configure anything. Click on Dashboards in the left sidebar, then click Infrastructure and Application Dashboard. You will see 8 panels showing real-time data.

Step 3: Explore Prometheus

```
http://localhost:9090
```

This is where metrics are stored. Try typing these queries in the query box:

```
# CPU usage percentage
100 - (avg(rate(node_cpu_seconds_total{mode="idle"}[5m])) * 100)

# Total HTTP requests to the demo app
http_requests_total

# Memory usage percentage
(1 - (node_memory_MemAvailable_bytes / node_memory_MemTotal_bytes)) * 100
```

Go to Status then Targets in the top menu. You should see 4 targets (prometheus, node-exporter, cadvisor, demo-app) all showing UP in green.

Step 4: Check Alertmanager

```
http://localhost:9093
```

This shows active alerts. If your machine is not under heavy load, you might not see any alerts firing. That is normal. The rules are configured to trigger only when thresholds are exceeded for a sustained period.

Step 5: Understand the Demo App

The demo app runs on port 8000 and generates its own traffic in the background. It hits a mix of fast, slow, and error-prone endpoints to create realistic dashboard data. You can also hit it manually:

```
curl http://localhost:8000/ # Normal response
curl http://localhost:8000/health # Health check
curl http://localhost:8000/slow # Simulates slow response (1-3 seconds)
curl http://localhost:8000/error # 70% chance of returning a 500 error
curl http://localhost:8000/metrics # Raw Prometheus metrics
```

The /metrics endpoint is what Prometheus scrapes every 15 seconds. It returns counters, histograms, and gauges in Prometheus format.

Step 6: Understanding the Alert Rules

The stack comes with 7 pre-configured alert rules:

Alert Name	Triggers When	Severity
HighCpuUsage	CPU above 80% for 5 minutes	Warning
HighMemoryUsage	Memory above 85% for 5 minutes	Warning
DiskSpaceLow	Disk below 15% for 10 minutes	Critical
ContainerDown	Any container stops running	Critical
ContainerHighCpu	Container CPU above 80% for 5 min	Warning
HighErrorRate	5xx errors exceed 5% of requests	Critical
SlowResponseTime	P95 latency above 2 seconds	Warning

Shutting Down

```
docker-compose down
```

This stops and removes all containers. Your data in Prometheus and Grafana is stored in Docker volumes, so it persists between restarts. To delete everything including data:

```
docker-compose down -v
```

Chapter 3
Terraform AWS Infrastructure

Production-ready AWS infrastructure defined entirely as code. Understand it without deploying it.

What This Project Is

This is a modular Terraform codebase that defines a complete AWS environment: a VPC with public and private subnets, an ECS Fargate service behind a load balancer, a bastion host for SSH access, and S3 with CloudFront for static content. You do not need an AWS account to study and understand this code. The architecture, module structure, and design decisions are the point.

What You Need

- Terraform 1.5 or newer installed (for validation only)
- No AWS account required to review and validate the code
- An AWS account and credentials only if you want to actually deploy

File Structure

```
terraform-aws-infra/
|-- main.tf # Root config, wires modules together
|-- variables.tf # All input variables with validation
|-- outputs.tf # Key outputs (IPs, DNS, bucket names)
|-- terraform.tfvars # Default values for staging
|-- modules/
| |-- vpc/ # VPC, subnets, IGW, NAT, routes
| | |-- main.tf
| | |-- variables.tf
| | +-- outputs.tf
| |-- security-groups/ # Bastion SG, ALB SG, ECS SG
| | +-- main.tf
| |-- ec2/ # Bastion host with user data
| | +-- main.tf
| |-- ecs/ # Fargate cluster, task def, ALB
| | +-- main.tf
| +-- static-hosting/ # S3 bucket, CloudFront CDN
| +-- main.tf
+-- .gitignore
```

Step 1: Clone and Explore

```
git clone https://github.com/Srj0210/terraform-aws-infra.git
cd terraform-aws-infra
```

Open main.tf first. This is the root configuration that calls all the modules. Each module block passes variables from one module to another. For example, the VPC module outputs subnet IDs, and those IDs are passed into the ECS module.

Step 2: Understand the Architecture

Here is what the code creates when deployed:

```
Internet --> CloudFront --> S3 Bucket (static assets)

Internet --> Internet Gateway --> ALB (public subnets)
|
v
ECS Fargate Tasks (private subnets)
|
v
NAT Gateway --> Internet (outbound only)

SSH --> Bastion Host (public subnet) --> Private resources
```

The key principle: ECS tasks run in private subnets with no public IP. The only way to reach them is through the Application Load Balancer. The bastion host is the only SSH entry point to anything in the VPC.

Step 3: Validate the Code

Even without AWS credentials, you can check if the Terraform code is syntactically correct:

```
terraform init
terraform validate
```

terraform init downloads the AWS provider plugin. terraform validate checks for syntax errors, missing variables, and type mismatches. If it says Success, the code is structurally sound.

> *Note: terraform validate does not check if the resources would actually work in AWS. For that, you need terraform plan with real credentials.*

Step 4: Review Each Module

VPC Module (modules/vpc/main.tf)

Creates the network foundation. A VPC with CIDR 10.0.0.0/16, two public subnets (10.0.1.0/24 and 10.0.2.0/24) and two private subnets (10.0.10.0/24 and 10.0.20.0/24) across two availability zones. An Internet Gateway handles public subnet traffic. A NAT Gateway in the first public subnet lets private subnets reach the internet for package updates without being directly exposed.

Security Groups Module

Three security groups with strict rules. The bastion SG allows SSH from anywhere (in production, restrict this to your IP). The ALB SG allows HTTP and HTTPS from the internet. The ECS SG only allows traffic from the ALB security group, not from any IP range. This means even other resources in the VPC cannot reach the containers directly.

EC2 Module

A single bastion host running Amazon Linux 2023. It has a user data script that installs Docker and Git on boot. IMDSv2 is enforced to prevent SSRF credential theft. The root volume is encrypted.

ECS Module

An ECS Fargate cluster with a task definition, service, and Application Load Balancer. Tasks run in private subnets with no public IP. The ALB sits in public subnets and forwards traffic to the containers. Container Insights is enabled for monitoring. CloudWatch Logs captures container output.

Static Hosting Module

An S3 bucket with versioning, encryption, and all public access blocked. CloudFront serves the content using Origin Access Control, which means even knowing the bucket name is not enough to access the files directly. The CDN handles HTTPS and caching globally.

Step 5: If You Want to Deploy (Optional)

If you have an AWS account and want to actually create these resources:

```
# Configure AWS credentials
aws configure

# Preview what will be created (dry run)
terraform plan

# Create everything
terraform apply

# When done, destroy everything to avoid charges
terraform destroy
```

Note: Running terraform apply will create real AWS resources that cost money. The NAT Gateway alone costs about 32 USD per month. Always run terraform destroy when you are done experimenting.

Chapter 4

Kubernetes Microservices Application

A 3-service microservice architecture running on a local Kubernetes cluster.

What This Project Is

Three containerized services running on Kubernetes, communicating through internal service discovery. A frontend gateway receives HTTP requests and calls a backend API. The backend processes requests and stores data in Redis. Each service has its own deployment, service, health probes, resource limits, and horizontal autoscaling. The whole thing runs locally on Minikube.

What You Need

- Docker installed
- Minikube installed (or Kind as an alternative)
- kubectl installed and configured

File Structure

```
k8s-microservices/
|-- services/
| |-- frontend/
| | |-- app.py # Flask gateway
| | |-- Dockerfile
| | +-- requirements.txt
| +-- backend/
| |-- app.py # Flask API + Redis
| |-- Dockerfile
| +-- requirements.txt
|-- k8s/
| |-- namespace.yaml # Isolated namespace
| |-- configmap.yaml # Environment config
| |-- frontend.yaml # Deployment + LoadBalancer
| |-- backend.yaml # Deployment + ClusterIP
| |-- redis.yaml # Deployment + ClusterIP
| |-- ingress.yaml # Path-based routing
| +-- hpa.yaml # Horizontal Pod Autoscalers
+-- deploy.sh # One-command deploy
```

Step 1: Start Minikube

```
minikube start --cpus=2 --memory=4096
minikube status
```

This creates a local single-node Kubernetes cluster. The cpus and memory flags allocate enough resources for all three services. Verify kubectl is connected:

```
kubectl cluster-info
kubectl get nodes
```

You should see one node with Ready status.

Step 2: Build Docker Images

```
git clone https://github.com/Srj0210/k8s-microservices.git
cd k8s-microservices

# Build both service images
docker build -t frontend:latest ./services/frontend/
docker build -t backend:latest ./services/backend/

# Load images into Minikube (important step)
minikube image load frontend:latest
minikube image load backend:latest
```

Note: The minikube image load step is critical. Without it, Kubernetes will try to pull the images from Docker Hub and fail because these are local images.

Step 3: Deploy Everything

You can either use the deploy script or do it manually:

Using the script:

```
chmod +x deploy.sh
./deploy.sh
```

Doing it manually step by step:

```
# Create namespace and config
kubectl apply -f k8s/namespace.yaml
kubectl apply -f k8s/configmap.yaml

# Deploy services (order matters: Redis first, then backend, then frontend)
kubectl apply -f k8s/redis.yaml
kubectl apply -f k8s/backend.yaml
kubectl apply -f k8s/frontend.yaml

# Apply autoscaling and ingress
kubectl apply -f k8s/hpa.yaml
kubectl apply -f k8s/ingress.yaml
```

Step 4: Verify Everything is Running

```
# Check all pods (should show 5 pods: 2 frontend, 2 backend, 1 redis)
kubectl get pods -n microservices

# Check services
```

```
kubectl get svc -n microservices

# Check HPA
kubectl get hpa -n microservices
```

All pods should show Running status with 1/1 in the READY column. If a pod shows 0/1, it means the readiness probe has not passed yet. Wait 15-20 seconds and check again.

Step 5: Access the Application

```
# Open the frontend service in your browser
minikube service frontend-service -n microservices
```

This command creates a tunnel and opens the frontend in your browser. You can also get the URL and use curl:

```
# Get the service URL
minikube service frontend-service -n microservices --url

# Test the endpoints
curl <URL>/
curl <URL>/api/data
```

The /api/data endpoint is the most interesting one. It calls the backend, which talks to Redis, and returns a combined response:

```
{
"source": "frontend",
"backend_response": {
"service": "backend",
"hostname": "backend-7d4f8b6c5-x2k9m",
"total_visits": 42,
"redis_connected": true
},
"status": "success"
}
```

Every time you hit this endpoint, the total_visits counter increases. The hostname field shows which backend pod handled the request. Hit it multiple times and you will see the hostname alternate between the two backend pods because of Kubernetes load balancing.

Step 6: Test Resilience

This is where it gets interesting. Let us break things on purpose and see how Kubernetes handles it.

Kill a pod and watch it recover:

```
# Delete a backend pod
kubectl delete pod -l app=backend -n microservices --wait=false

# Watch it get replaced immediately
kubectl get pods -n microservices -w
```

Kubernetes sees that the desired replica count is 2 but only 1 pod exists, so it immediately creates a new one. During this time, the other backend pod handles all traffic. This is self-healing in action.

Scale manually:

```
# Scale frontend to 4 replicas
kubectl scale deployment frontend -n microservices --replicas=4

# Verify
kubectl get pods -n microservices -l app=frontend
```

Check logs:

```
# See logs from all backend pods
kubectl logs -l app=backend -n microservices --tail=20

# Follow logs in real time
kubectl logs -l app=frontend -n microservices -f
```

Step 7: Clean Up

```
# Delete everything in the namespace
kubectl delete namespace microservices

# Or stop Minikube entirely
minikube stop
minikube delete
```

Appendix

Prerequisites and Installation

How to install every tool you need on your machine.

Docker

```
# Ubuntu/Debian
sudo apt update
sudo apt install docker.io docker-compose -y
sudo usermod -aG docker $USER
# Log out and back in after this

# Windows/Mac
# Download Docker Desktop from https://docker.com/products/docker-desktop

# Verify
docker --version
docker-compose --version
```

Python 3.11

```
# Ubuntu/Debian
sudo apt install python3 python3-pip -y

# Windows
# Download from https://python.org/downloads

# Verify
python3 --version
pip3 --version
```

Terraform

```
# Ubuntu/Debian
wget -O- https://apt.releases.hashicorp.com/gpg | sudo gpg --dearmor -o
/usr/share/keyrings/hashicorp-archive-keyring.gpg
echo "deb [signed-by=/usr/share/keyrings/hashicorp-archive-keyring.gpg]
https://apt.releases.hashicorp.com $(lsb_release -cs) main" | sudo tee
/etc/apt/sources.list.d/hashicorp.list
sudo apt update && sudo apt install terraform -y

# Windows
# Download from https://developer.hashicorp.com/terraform/downloads

# Verify
terraform --version
```

kubectl

```
# Ubuntu/Debian
curl -LO "https://dl.k8s.io/release/$(curl -L -s
https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl"
sudo install -o root -g root -m 0755 kubectl /usr/local/bin/kubectl

# Windows
# Download from https://kubernetes.io/docs/tasks/tools/install-kubectl-windows/

# Verify
kubectl version --client
```

Minikube

```
# Ubuntu/Debian
curl -LO https://storage.googleapis.com/minikube/releases/latest/minikube-linux-amd64
sudo install minikube-linux-amd64 /usr/local/bin/minikube

# Windows
# Download from https://minikube.sigs.k8s.io/docs/start/

# Verify
minikube version
```

Git

```
# Ubuntu/Debian
sudo apt install git -y

# Windows
# Download from https://git-scm.com/downloads

# Verify
git --version
```

End of Guide

Built by Suraj Maitra

github.com/Srj0210 | surajmaitra@gmail.com